SIMATS ENGINEERING
DESCRIPTIVE WRITTEN EXAM – ANSWERS

COURSE NAME: Software Testing **STUDENT NAME:** B.Satya Hyndavi

COURSE CODE: CSA3712 **REG.NO:**192311178

SECTION A – ALL 20 QUESTIONS

1. A banking application team is facing frequent failures due to unstructured testing practices. As a test engineer, apply the concept of testing as an engineering activity to redesign the testing approach. Explain how systematic test design, tools, and standards can be introduced. Illustrate how this approach improves defect detection and quality. Suggest suitable techniques to replace ad-hoc testing. Justify your solution with a practical example.

2. An ATM system is producing inconsistent results due to lack of a defined testing process. As a tester, demonstrate how you would implement a structured testing process across SDLC phases. Explain each stage and its relevance to quality improvement. Apply your approach to ATM withdrawal functionality. Discuss how process consistency reduces defects. Conclude with expected outcomes.

3. A web application shows mismatch between expected and actual outputs during login and transaction operations. Apply basic testing definitions to identify errors, defects, and failures in this scenario. Illustrate their relationships with suitable examples. Explain how improper understanding of these terms affects testing. Suggest how to improve defect reporting clarity. Justify your answer with examples.

4. A software project team observes that most defects are concentrated in a few modules, leading to delays. Apply software testing principles to address this issue. Explain how defect clustering and other principles guide test planning. Illustrate how test cases can be prioritized effectively. Discuss how applying these principles improves testing efficiency. Provide a real-time example.

5. A company maintains a defect repository but fails to utilize it effectively, resulting in repeated defects across releases. As a tester, apply defect classification and repository concepts to improve the situation. Explain how defects originate and how they can be categorized. Illustrate how

repository data can support test design and prevention. Suggest improvements to avoid recurring defects. Conclude with expected quality improvements.

6. A mobile banking application frequently crashes during fund transfer operations. As a tester, apply software testing principles to identify the root cause of failures. Explain how systematic testing can reduce such issues. Illustrate your approach with suitable examples. Suggest improvements in testing practices to avoid future failures.

7. A development team confuses debugging with testing, leading to ineffective defect resolution. Apply the concepts of testing and debugging to differentiate both processes. Explain their roles in software quality. Illustrate how improper usage affects defect detection. Suggest a structured approach to integrate both effectively.

8. An e-commerce platform shows inconsistent behavior across browsers. Apply testing principles to design a strategy for cross-browser testing. Explain how defects originate in such scenarios. Illustrate how systematic testing improves reliability. Suggest tools and techniques for better coverage.

9. A payroll system produces incorrect salary calculations due to logical errors in code. Apply defect classification techniques to categorize these defects. Explain how understanding defect classes improves testing. Illustrate how such defects can be prevented in future releases. Suggest improvements in testing strategy.

10. A software team skips proper test case design and directly executes random tests. Apply testing as an engineering activity to redesign the testing process. Explain the importance of structured test case design. Illustrate how this improves defect detection. Suggest suitable techniques to enhance test effectiveness.

11. A healthcare application shows data inconsistency across modules. Apply testing principles to analyze the issue. Explain how defects may originate during development. Illustrate how systematic testing ensures data integrity. Suggest improvements in defect tracking and reporting.

12. A company faces repeated defects in its software releases despite testing efforts. Apply the concept of defect repository to analyze the issue. Explain how repository data supports test design. Illustrate how defect patterns can be identified. Suggest improvements to prevent recurring defects.

13. A web-based learning system fails during peak usage due to untested scenarios. Apply software testing principles to identify gaps in testing. Explain how test planning and coverage impact performance. Illustrate how better test strategies improve system reliability. Suggest enhancements to avoid such failures.

14. A team observes that testers are not involved in early development stages. Apply the role of tester in SDLC to improve this situation. Explain how early involvement impacts quality. Illustrate benefits using a practical example. Suggest changes in team collaboration practices.

15. A system produces correct outputs in some cases but fails in others due to incomplete test coverage. Apply testing principles to identify gaps. Explain how proper coverage ensures reliability. Illustrate how to improve test case design. Suggest techniques for better validation.

16. A logistics application shows delay in defect identification due to poor reporting practices. Apply testing concepts to improve defect reporting. Explain how clear definitions of defects help in tracking. Illustrate how structured reports enhance communication. Suggest improvements in reporting standards.

17. A software team struggles to maintain consistency in testing across modules. Apply process-based testing approaches to solve the issue. Explain how standardized processes improve quality. Illustrate your approach with an example. Suggest methods to ensure consistency.

18. A system fails after deployment due to untested edge cases. Apply testing principles to identify why such cases were missed. Explain how boundary testing helps in such scenarios. Illustrate how to include edge cases in testing. Suggest improvements in test design.

19. A company lacks proper documentation of defects, making it difficult to track issues. Apply defect repository concepts to address this problem. Explain how documentation improves defect management. Illustrate how structured repositories support analysis. Suggest improvements in documentation practices.

20. A banking system shows repeated failures in transaction processing due to improper testing practices. Apply software testing principles to redesign the testing approach. Explain how structured testing improves quality. Illustrate your solution with examples. Suggest techniques to enhance defect detection.

SECTION B – ANSWERS

Q1. Answer

A banking application should be tested as an engineering activity rather than by trying random inputs. The process should be planned, repeatable and measurable.

1. Redesigned approach

- Understand requirements and identify important banking functions such as login, balance enquiry, transfer and payment.
- Prepare a test plan with scope, risks, test levels, responsibilities and entry/exit conditions.
- Design test cases before execution using requirements as the basis.
- Record actual results and compare them with expected results.
- Log every defect clearly and retest after the fix.

2. Systematic test design

Use equivalence partitioning for valid and invalid inputs, boundary value analysis for limits such as transfer amount, and decision-based test cases for conditions such as sufficient or insufficient balance. Regression tests should be maintained for important functions.

3. Tools and standards

A defect-tracking tool can maintain defect status, severity and evidence. Test management tools can store test cases and results. A common format for test cases and defect reports gives consistency between testers.

4. Replacing ad-hoc testing

Instead of random testing, use requirement-based testing, risk-based prioritization, boundary testing, equivalence partitioning and regression testing.

Practical example: For a transfer of ₹10,000, test valid transfer, zero amount, negative amount, amount above the allowed limit, insufficient balance and invalid beneficiary details. This is more effective than trying random amounts.

Conclusion: An engineering-based approach gives planned coverage, repeatable testing and better defect tracking. It therefore improves defect detection and the overall quality of the banking application.

Q2. Answer

A structured testing process should be followed throughout the SDLC so that defects are prevented or detected early instead of being found only after deployment.

1. Requirements phase

The tester reviews ATM requirements such as card validation, PIN verification, balance checking and withdrawal limits. Ambiguous requirements are reported early.

2. Design phase

Test scenarios are prepared from the design. For withdrawal, conditions such as available balance, daily limit and ATM cash availability are identified.

3. Development phase

Developers perform unit testing for individual functions such as PIN validation and balance calculation. Testers prepare detailed test cases in parallel.

4. Integration and system testing

The complete withdrawal flow is tested: insert card → enter PIN → select withdrawal → enter amount → validate balance/limit → dispense cash → update balance → print/display confirmation.

5. Acceptance testing

The withdrawal function is checked against business requirements using realistic scenarios.

Example test cases:

- Valid card + valid PIN + sufficient balance → cash should be dispensed.
- Invalid PIN → transaction should be rejected.
- Amount greater than balance → withdrawal should fail.
- Amount greater than daily limit → transaction should fail.
- Insufficient cash in ATM → appropriate message should be shown.

A defined process makes testing repeatable. The same important checks are not forgotten when different testers work on different releases. It also makes defects easier to trace to requirements and test cases.

Conclusion: Applying testing at every SDLC stage reduces late defects, improves consistency and increases confidence that ATM withdrawal works correctly under normal and exceptional conditions.

Q3. Answer

The terms error, defect and failure describe different stages of a software problem.

1. Error

An error is a human mistake made during requirements, design, coding or another development activity. For example, a developer may misunderstand the rule that a password must contain a special character.

2. Defect

When the mistake is introduced into the software or its related work product, it becomes a defect. For example, the login code may incorrectly accept a password that does not satisfy the requirement.

3. Failure

A failure occurs when the running software produces an incorrect result or behavior. If the application accepts the invalid password during actual execution, the user observes a failure.

Relationship:

Error → Defect → Failure

The same idea applies to a transaction. If a developer misunderstands the transaction rule, the code may contain a defect, and during execution the application may show an incorrect transaction amount.

Why clear understanding matters:

If testers call every problem simply an “error,” developers may not know whether the issue is in the requirement, code or observed behavior. This can delay root-cause analysis and correction.

Improving defect reporting:

A clear report should contain:

- Defect ID
- Module/function
- Preconditions
- Steps to reproduce
- Expected result
- Actual result
- Severity and priority
- Screenshot or evidence

- Environment details

Example: "Transaction page displays ₹5,000 as the transferred amount although ₹3,000 was entered" is clearer than "transaction error."

Conclusion: Using error, defect and failure correctly improves communication, makes reports precise and helps the team identify and fix problems faster.

Q4. Answer

Defect clustering means that a small number of modules may contain a large proportion of defects. Therefore, testing effort should not always be distributed equally.

1. Applying defect clustering

First, examine previous defect records and identify modules with frequent or severe defects. These modules become high-risk areas for additional testing.

2. Other principles for planning

- Test important functions early.
- Concentrate more effort on complex and frequently changed modules.
- Use previous defect history to identify risky areas.
- Perform regression testing after fixes.
- Do not assume that defect-free areas require no testing.

3. Prioritizing test cases

For an application containing Login, Payment, Profile and Help modules, suppose Payment has most previous defects. Payment test cases should be executed first because a failure there has greater business impact.

A simple priority can be:

High: payment, transaction and security functions.

Medium: profile and account-management functions.

Low: less critical informational functions.

Example: In an online shopping application, previous releases may show many defects in the checkout and payment modules. Testers can execute valid payment, failed payment, duplicate payment, timeout and refund scenarios before spending equal effort on low-risk pages.

Benefits:

- High-risk defects are found earlier.
- Testing time is used more effectively.
- Regression effort becomes focused.
- Release decisions can be based on actual risk and defect history.

Conclusion: Defect clustering does not mean ignoring other modules. It means using available evidence to prioritize testing intelligently. This increases efficiency while protecting the most important functions.

Q5. Answer

A defect repository is useful only when the organization learns from the defects stored in it. Repeated defects indicate that the repository is being treated as a record rather than as a source for improvement.

1. How defects originate

Defects may originate from misunderstood requirements, incorrect design, coding mistakes, integration problems, incorrect data or inadequate testing.

2. Classification

Defects can be categorized by:

- Requirement
- Design
- Coding
- Interface/integration
- Data
- Environment

They can also be classified by severity, priority, module and status.

3. Using repository data

The tester should review previous defects before designing new tests. If several releases contain defects in a calculation module, additional boundary, negative and regression tests should be created for that module.

Example:

Suppose a banking application repeatedly had defects in transaction reversal. The repository shows that most failures occurred after timeout or network interruption. The next test cycle should therefore include timeout, retry, duplicate request and reversal scenarios.

4. Preventing recurrence

- Add regression tests for corrected defects.
- Review recurring defect patterns.
- Identify the root cause instead of only fixing symptoms.
- Update test cases when a new defect is discovered.
- Share important lessons with developers and testers.

Expected improvement:

The team can detect similar defects earlier, avoid repeating the same mistakes and gradually strengthen the regression suite.

Conclusion: A well-maintained repository becomes a learning resource. Using its history in test design changes testing from a reactive activity into a preventive activity.

Q6. Answer

Frequent crashes during fund transfer indicate that the testing process needs to cover the complete transaction flow and abnormal conditions.

1. Identify the failure

First reproduce the crash consistently. Record the account type, amount, network condition, device, operating system and exact steps before the crash.

2. Find possible root causes

The problem may be caused by invalid input handling, insufficient balance, network interruption, session timeout, concurrency, incorrect transaction-state handling or an application defect.

3. Systematic testing approach

Prepare test cases for:

- Valid transfer.
- Zero and negative amount.
- Amount greater than balance.

- Amount at and above transaction limits.
- Network loss during transfer.
- App closed during transfer.
- Repeated transfer request.
- Session timeout.
- Different devices and network conditions.

4. Example

If the application crashes when the network is disconnected after the user presses “Transfer,” the tester should reproduce the condition, capture logs and report the exact steps. After the fix, the same test becomes a regression test.

Improvements:

- Use risk-based testing for money-transfer functions.
- Automate stable regression scenarios.
- Test negative and boundary conditions.
- Include device and network variations.
- Review crash reports and recurring defects.

Conclusion: Systematic testing does not only check whether a normal transfer works. It also checks what happens when conditions change. This helps identify the actual cause of crashes and prevents similar failures in future releases.

Q7. Answer

Testing and debugging are related but they are not the same activity.

Testing - **Debugging**

Finds defects or failures - Investigates and fixes the cause

Checks software against expected behavior - Examines code, data or logic

Usually performed by testers and developers - Mainly performed by developers

Produces evidence of a problem - Produces a correction

Role in quality:

Testing provides information about whether the software meets requirements. Debugging uses that information to locate and correct the underlying problem.

Example:

Suppose a payroll application displays an incorrect salary. The tester executes the test case, compares expected and actual values and reports the failure. The developer then traces the calculation logic, finds the incorrect formula and fixes it. The tester retests the corrected version.

Effect of confusion:

If a tester tries to fix code instead of reporting the failure clearly, the problem may not be independently verified. Similarly, if developers only debug without proper testing, some failures may remain undiscovered.

Structured integration:

1. Tester executes planned test cases.
2. Failure is documented with evidence.
3. Developer analyzes the defect.
4. Developer fixes the root cause.
5. Tester performs retesting.
6. Regression testing checks that existing functions still work.
7. Defect is closed only after verification.

Conclusion: Testing identifies whether there is a problem, while debugging helps find and remove its cause. Using both in sequence makes defect resolution more effective.

Q8. Answer

Cross-browser testing is required because the same web application may behave differently in different browsers, versions or environments.

1. Identify the required environment

Prepare a browser matrix based on the browsers and versions that users are expected to use. Include desktop and, where relevant, mobile browsers.

2. Main areas to test

- Login and session handling.
- Page layout and responsive design.
- Buttons, forms and navigation.
- JavaScript functionality.
- File upload/download.
- Transactions and calculations.
- Cookies and storage.
- Error messages.

3. Why defects originate

Differences in browser engines, CSS support, JavaScript behavior, screen sizes or browser versions can cause inconsistent behavior.

Example:

A checkout button may work correctly in one browser but become misaligned or inactive in another because of a CSS or JavaScript compatibility issue.

4. Testing strategy

Start with the most widely used browser combinations. Execute common functional tests on each environment and then perform targeted tests for browser-specific risks.

Tools and techniques:

- Browser developer tools for investigation.
- Cross-browser testing platforms for multiple environments.
- Automated regression tests for repeatable checks.
- Responsive testing for different screen sizes.

Systematic testing improves reliability because the team does not assume that one browser result represents every environment.

Conclusion: A planned browser matrix, prioritized test cases and repeatable regression tests provide better coverage and reduce compatibility defects before release.

Q9. Answer

Incorrect salary calculations are primarily logical or functional defects because the software does not apply the required calculation rules correctly.

1. Defect classification

The defect may be classified as:

- Coding/logic defect – incorrect formula or condition.
- Requirement defect – if the salary rule itself was misunderstood.
- Data defect – if incorrect salary inputs or rates are stored.
- Integration defect – if payroll data received from another module is incorrect.

2. Why classification matters

Classification helps the team understand where defects originate and which testing techniques should be strengthened. A repeated logic defect suggests a need for stronger calculation and boundary testing.

Example:

If basic salary is ₹30,000 and the application incorrectly calculates an allowance using the wrong percentage, the tester should compare the expected formula with the actual result.

Useful test cases:

- Minimum salary.
- Normal salary.
- Maximum salary.
- Zero allowance.
- Maximum allowance.
- Different employee categories.
- Deduction and tax combinations.

Prevention:

- Review calculation requirements with developers.
- Use independent expected-value calculations in test cases.
- Perform boundary and equivalence testing.
- Add corrected calculation cases to regression testing.
- Review defects to identify repeated logic mistakes.

Conclusion: Proper classification helps identify the source and pattern of salary defects. Combining clear requirements, calculation-focused test cases and regression testing can prevent the same logical errors in future releases.

Q10. Answer

Random testing without test case design makes it difficult to know what has been tested and what has been missed. Testing should therefore be treated as an engineering activity.

1. Redesign the process

- Study requirements.
- Identify functions and risks.
- Prepare test scenarios.
- Convert scenarios into test cases with inputs, steps and expected results.
- Prioritize cases.
- Execute and record results.
- Report defects.
- Retest fixes and perform regression testing.

2. Importance of structured test cases

A structured test case gives the tester a clear expected result. It also makes testing repeatable and allows another tester to reproduce the same test.

Example:

For a login page:

TC01 – valid username and password → login successful.

TC02 – valid username and invalid password → login rejected.

TC03 – blank username → validation message.

TC04 – blank password → validation message.

TC05 – both blank → appropriate validation.

3. Suitable techniques

- Equivalence partitioning.
- Boundary value analysis.
- Decision table testing.
- State-based testing.
- Negative testing.
- Risk-based prioritization.
- Regression testing.

Benefits:

Structured design improves coverage, reduces missed scenarios and provides traceability between requirements and tests.

Conclusion: Random execution may find some defects by chance, but engineering-based test design provides planned and measurable coverage. It makes testing more effective and improves confidence in the software.

Q11. Answer

Data inconsistency across healthcare modules is a serious quality issue because different modules may show different values for the same patient or transaction.

1. Possible origins

Defects may originate from incorrect requirements, database mapping, data conversion, interface logic, synchronization problems or coding errors.

2. Systematic analysis

Identify the source record and compare it with the values displayed in connected modules. Trace the data through input, storage, processing and output.

3. Test approach

- Verify data entered in the source module.
- Check database/storage values where appropriate.

- Verify data transfer between modules.
- Compare values in receiving modules.
- Test update and delete operations.
- Test repeated updates and concurrent changes.
- Check invalid and missing data.

Example:

If a patient's updated contact number is changed in the registration module but an appointment module displays the old number, integration or synchronization testing should identify where the value becomes inconsistent.

Defect tracking should include:

Module, data involved, steps to reproduce, expected value, actual value, severity, environment and evidence.

Improvements:

- Maintain clear data requirements.
- Add integration test cases.
- Use regression tests for corrected data issues.
- Track defects with unique IDs and status.
- Review recurring data defects.

Conclusion: Systematic data-flow and integration testing helps ensure that the same information remains accurate across modules. Clear defect reporting also makes the source of inconsistency easier to identify and fix.

Q12. Answer

Repeated defects across releases indicate that previous defect information is not being converted into stronger testing.

1. Use the defect repository

The repository should contain defect ID, module, description, severity, root cause, status, release and corrective action. This allows the team to study past failures.

2. Support for test design

Before a new release, testers should review defects from earlier releases. Every important corrected defect should be considered for regression testing.

3. Identifying patterns

Patterns can be found by grouping defects by:

- Module.
- Defect type.
- Severity.
- Root cause.
- Release.
- Frequency.

Example:

If the repository shows repeated defects in the payment module after changes to transaction validation, that module should receive additional regression and boundary tests in the next release.

4. Preventing recurrence

- Convert important defects into permanent regression test cases.
- Perform root-cause analysis.
- Review modules with repeated failures.
- Improve test coverage around risky changes.
- Share defect trends with developers and testers.

A repository therefore becomes more than a list of old issues. It provides evidence for deciding where testing effort should be concentrated.

Conclusion: Studying historical defect patterns helps the team predict risky areas and strengthen test design. This reduces repeated defects and improves quality across releases.

Q13. Answer

A learning system that fails during peak usage shows that testing was focused mainly on normal conditions and did not sufficiently consider performance-related scenarios.

1. Identify testing gaps

The team should check whether it tested:

- Expected number of users.
- Peak concurrent users.
- Large numbers of requests.
- Login during high traffic.
- Course access during peak periods.
- Database response under load.
- Recovery after high load.

2. Test planning and coverage

The test plan should define performance risks and the expected user load. Coverage should include normal, high and peak usage rather than only one user or a small test group.

3. Example

Suppose 100 users can access the learning system normally, but the application is expected to support 1,000 users during an examination. Testing only with 100 users leaves an important gap.

4. Better strategy

Use performance/load testing for increasing numbers of users, monitor response time and errors, and identify the point at which performance becomes unacceptable. Functional regression testing should continue alongside performance-focused testing.

5. Reliability improvements

- Include peak-load scenarios in the test plan.
- Test realistic user behavior.
- Repeat performance tests after major changes.
- Track response-time and failure trends.
- Add high-risk performance scenarios to regression checks where practical.

Conclusion: Test coverage must reflect real usage conditions. Including peak and high-load scenarios before deployment can reveal performance weaknesses and improve system reliability during actual high-demand periods.

Q14. Answer

Testers should be involved early in the SDLC instead of being brought in only after development is complete.

1. Early involvement

During requirements analysis, testers can review requirements for ambiguity, missing conditions and testability. During design, they can identify important scenarios and risks. While developers code, testers can prepare test cases.

2. Benefits

- Defects can be prevented earlier.
- Requirements become clearer.
- Test cases can be prepared before execution.
- Risks are identified sooner.
- Rework and late fixing can be reduced.

3. Practical example

Suppose an ATM requirement says, “Allow cash withdrawal within the daily limit.” A tester may ask what should happen when the amount is exactly equal to the limit, when the balance is insufficient, or when the ATM has insufficient cash. Clarifying these cases early prevents misunderstandings.

4. Collaboration changes

- Include testers in requirement reviews.
- Conduct joint discussions between developers and testers.
- Share acceptance criteria early.
- Review test scenarios before coding is complete.
- Maintain clear communication about changes.

Testing then becomes a continuous quality activity rather than a final checking stage.

Conclusion: Early tester involvement improves understanding, exposes missing requirements and allows test planning to begin sooner. This can prevent defects instead of finding all of them after development.

Q15. Answer

Incomplete coverage means that some valid, invalid or boundary conditions have not been tested. A system may therefore appear correct for common cases while failing under other conditions.

1. Identify coverage gaps

Compare requirements with existing test cases. Check whether each major function, condition, input class and important exception has at least one suitable test.

2. Improve test design

Use:

- Equivalence partitioning to cover groups of similar inputs.
- Boundary value analysis for minimum and maximum values.
- Decision tables for combinations of conditions.
- State-based testing for different system states.
- Negative testing for invalid inputs.
- Regression testing after changes.

Example:

If an application accepts an age from 18 to 60, testing only age 25 is insufficient. Useful boundary cases include 17, 18, 19, 59, 60 and 61.

3. Coverage and reliability

Better coverage increases the chance of finding defects in conditions that users may actually encounter. It does not mean testing every possible input, but selecting representative and high-risk cases systematically.

4. Validation improvement

Map test cases to requirements and review uncovered areas before release. Prioritize high-risk and frequently changed functions.

Conclusion: Proper coverage gives stronger evidence that the software works across important conditions. Structured test design is more reliable than testing only a few normal examples.

Q16. Answer

Poor defect reporting delays identification and resolution because developers may not have enough information to reproduce the problem.

1. Clear defect definition

A defect report should clearly state what the software did incorrectly compared with what was expected.

2. Recommended report structure

- Defect ID.
- Short and specific title.
- Module/function.
- Environment and version.
- Preconditions.
- Steps to reproduce.
- Expected result.
- Actual result.
- Severity and priority.
- Screenshot, log or other evidence.
- Current status.

3. Example

Weak report: “Delivery screen is not working.”

Better report: “Delivery screen shows an incorrect ETA after entering a valid destination. Steps: login → open delivery → enter destination → select date → submit. Expected: correct ETA displayed. Actual: previous ETA remains.”

4. Benefits

Structured reports reduce repeated questions, help developers reproduce the issue and make tracking easier. Clear severity and priority also help the team decide which defects require immediate attention.

Reporting standards should be common across the team so every tester provides the same essential information.

Conclusion: Good reporting is part of effective testing. A clear, reproducible defect report improves communication, speeds up diagnosis and helps the team track the issue until verified closure.

Q17. Answer

Consistency across modules can be improved by using a common, process-based testing approach rather than allowing every tester to follow a different method.

1. Standard process

For each module, follow the same basic flow:

Requirement review → test planning → test case design → test data preparation → execution → defect reporting → retesting → regression → closure.

2. Standard documents

Use common formats for test cases and defect reports. Test cases should contain steps, input, expected result and actual result. Defect reports should use agreed fields such as severity, priority and status.

3. Example

Suppose Login, Payment and Profile are tested by different testers. All three testers should use the same test-case structure and defect-reporting rules. Payment may receive additional risk-based cases, but the basic process remains consistent.

4. Methods to maintain consistency

- Create testing checklists.
- Use common templates.
- Conduct peer reviews.
- Maintain reusable regression suites.
- Record process guidelines.
- Review test results regularly.
- Track whether required test activities were completed.

Benefits:

Standardization makes results easier to compare, reduces missed activities and allows another tester to continue the work without changing the basic process.

Conclusion: Process-based testing does not mean every module receives exactly the same tests. It means the testing method is consistent while test depth is adjusted according to risk and module behavior.

Q18. Answer

Edge cases are conditions at or near unusual or limiting values. They are often missed when testing focuses only on normal user behavior.

1. Why edge cases are missed

They may not be clearly identified in requirements, testers may use only common inputs, or the test design may not include boundaries and exceptional conditions.

2. Boundary testing

Boundary value analysis focuses on values at the minimum and maximum limits and values just below or above them.

Example:

If a system accepts a quantity from 1 to 100, useful tests include:

- 0
- 1
- 2
- 99
- 100
- 101

3. Including edge cases

During test design, identify every important limit, state change and exceptional condition. Add corresponding test cases before execution.

Other edge cases can include:

- Empty input.
- Maximum-length input.
- Duplicate request.
- Network interruption.

- Session timeout.
- Very large or very small values.

4. Improvements

Review requirements specifically for boundaries, use equivalence partitioning and boundary analysis, and add every important discovered edge case to the regression suite.

Conclusion: Boundary and edge-case testing helps reveal defects that normal test cases may miss. Including these cases systematically before release reduces the chance of unexpected production failures.

Q19. Answer

A defect repository should provide a complete and organized history of software problems. If defects are poorly documented, the team cannot easily track, analyze or learn from them.

1. Information to document

Each defect should have:

- Unique ID.
- Module.
- Short description.
- Steps to reproduce.
- Expected result.
- Actual result.
- Severity and priority.
- Environment/version.
- Evidence.
- Assigned person.
- Status.
- Root cause and resolution, when available.

2. How documentation improves management

Clear documentation allows the team to know what was reported, who is handling it, whether it was fixed and whether the fix was verified.

3. Supporting analysis

A structured repository allows defects to be grouped by module, type, severity, release and root cause. This can reveal recurring problem areas.

Example:

If several defects are recorded against a transaction module, the team can review the pattern and increase regression and risk-based testing for that module.

4. Documentation improvements

Use a standard defect template, mandatory fields, consistent severity definitions and regular repository reviews. Closed defects should remain available as historical information rather than being discarded.

Conclusion: Proper defect documentation improves traceability, communication and analysis. A structured repository also turns past defect information into useful input for future testing and prevention.

Q20. Answer

Repeated transaction failures in a banking system require a structured testing approach because transactions involve important data, business rules and failure conditions.

1. Understand requirements and risks

Identify transaction rules such as valid accounts, balance requirements, limits, duplicate prevention, transaction status and failure handling.

2. Design systematic tests

Prepare normal, negative, boundary and exception cases. Examples include:

- Valid transaction with sufficient balance.
- Insufficient balance.
- Invalid account details.
- Amount at the allowed limit.
- Amount above the limit.
- Network interruption during transaction.
- Repeated submission.
- Session timeout.

- Failed transaction followed by retry.

3. Execute and track

Record expected and actual results for every test. Any mismatch should be reported with steps, evidence, severity and environment.

4. Regression testing

When a transaction defect is fixed, rerun the failed test and related transaction cases. This ensures that the correction has not introduced another problem.

5. Improve defect detection

Use risk-based prioritization, equivalence partitioning, boundary value analysis, negative testing and regression testing. Previous defects should also be reviewed so recurring problem areas receive additional attention.

Example:

If a duplicate transfer occurs when a user presses the transfer button twice during a slow network response, the team should reproduce the condition and add it permanently to regression testing.

Conclusion: Structured testing gives better coverage of both normal and exceptional transaction conditions. It reduces repeated failures, improves defect detection and provides stronger confidence before deployment.

Code of Conduct:

I ____**G Bhumika/192312237**____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

G Bhumika